

開始使用 Angular Signals

來源：<https://codelabs.developers.google.com/angular-signals?hl=zh-tw>

步驟數：7

隆重推出信號，這是 Angular 的全新反應式模型。信號可提供更多高品質工具，讓您在設定應用程式效能時，能以更精細的程度做出反應，同時提供明確的指引和防護措施，確保應用程式效能良好。瞭解如何立即開始探索 Angular v16 的開發人員預覽版。

目錄

1. 1. 事前準備
2. 2. 取得程式碼
3. 3. 建立基準
4. 4. 定義第一個信號()
5. 5. 定義第一個 `computed()`
6. 6. 新增第一個 `effect()`
7. 7. 恭喜！

步驟 1 · 約 1 分鐘

1. 事前準備



Angular Signals 為您熟悉且喜愛的 Angular 導入三種反應式基本型別，簡化開發作業，並協助您預設建構速度更快的應用程式。

建構項目

- 您將瞭解 Angular Signals 導入的三個反應式基本型別：`signal()`、`computed()` 和 `effect()`。
- 使用 Angular Signals 驅動 Angular Cipher 遊戲。密碼是加密及解密資料的系統。在這個遊戲中，使用者可以拖曳線索解開密碼，解讀秘密訊息、自訂訊息，以及分享網址給朋友，傳送秘密訊息。



必要條件

- 熟悉 Angular 和 Typescript
 - 建議：觀看「[使用信號重新思考反應式程式設計](#)」，瞭解 Angular Signals 程式庫
-

步驟 2 · 約 1 分鐘

2. 取得程式碼

本專案所需的一切都在 Stackblitz 中。建議使用 Stackblitz 完成本程式碼研究室。您也可以複製程式碼，並在慣用的開發環境中開啟。

開啟 Stackblitz 並執行應用程式

如要開始使用，請在慣用的網路瀏覽器中開啟 Stackblitz 連結：

1. 開啟新的瀏覽器分頁，然後前往 <https://stackblitz.com/edit/io-signals-codelab-starter?file=src%2Fcipher%2Fservice.cipher.ts,src%2Fsecret-message%2Fservice.message.ts&service.message.ts>
2. 將 Stackblitz 分叉，建立自己的可編輯工作區。Stackblitz 應會自動執行應用程式，這樣就大功告成了！

替代做法：複製存放區並提供應用程式

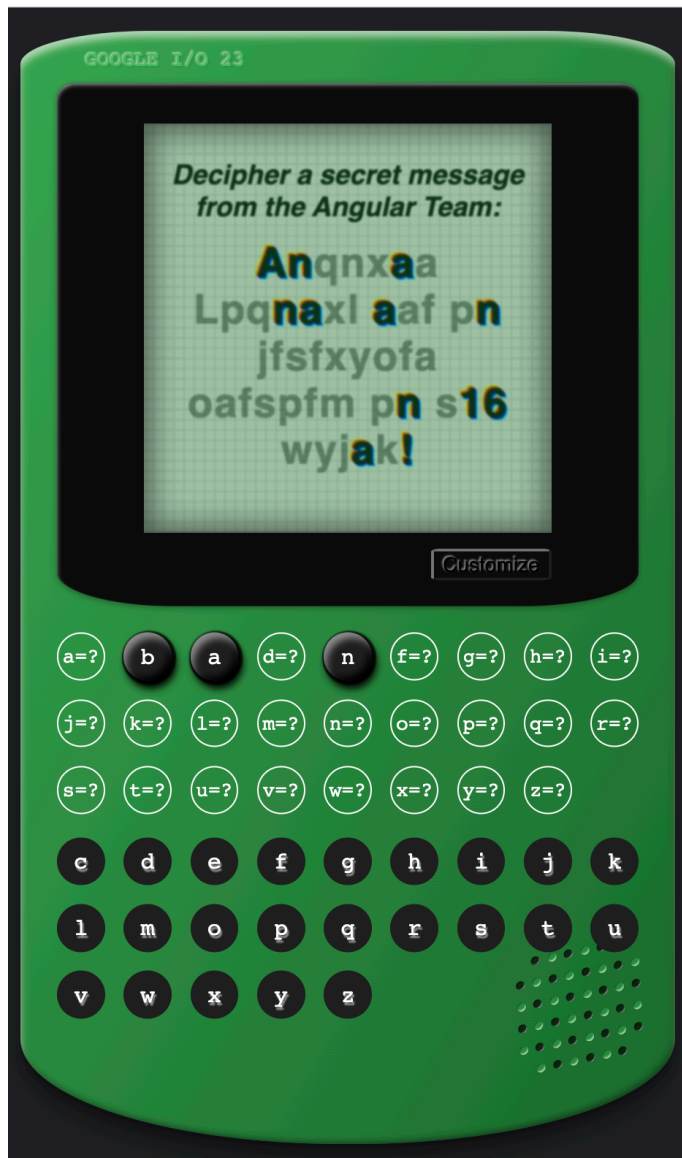
您也可以使用 VSCode 或本機 IDE 完成本程式碼研究室：

1. 開啟新的瀏覽器分頁，然後前往 <https://github.com/angular/codelabs/tree/signals-get-started>。
 2. 分叉並複製存放區，然後使用 `cd codelabs/` 指令移至存放區。
 3. 使用 `git checkout signals-get-started` 指令查看範例程式碼分支版本。
 4. 在 VSCode 或慣用的 IDE 中開啟程式碼。
 5. 如要安裝執行伺服器所需的依附元件，請使用 `npm install` 指令。
 6. 如要執行伺服器，請使用 `ng serve` 指令。
 7. 開啟瀏覽器分頁，前往 <http://localhost:4200>。
-

步驟 3 · 約 3 分鐘

3. 建立基準

您的起點是 Angular Cipher 遊戲，但目前還無法運作。Angular Signals 將支援遊戲功能。



如要開始使用，請先瀏覽您要建構的完成版：[Angular Signals Cypher](#)。

1. 在畫面上查看編碼訊息。
2. 將鍵盤中的字母按鈕拖曳到密碼中，解開密文並解碼秘密訊息。
3. 成功後，請查看訊息更新情形，瞭解如何解碼更多秘密訊息。
4. 按一下「自訂」即可變更「寄件者」和「訊息」，然後按一下「建立並複製網址」，即可在畫面上查看值和網址變更。
5. 額外好康：將網址複製並貼到新分頁，或分享給朋友，看看網址中儲存了哪些寄件者和訊息資訊。

GOOGLE I/O 23

*Decipher a secret message from
the Angular Team:*

Angugaq
Dragnagd aqb rn
ibobgjybq
yqborbl rn o16
zjiam!

Customize



4. 定義第一個信號()

訊號是可告知 Angular 何時變更的值。部分信號可直接變更，其他信號則會根據其他信號的值計算值。信號會共同建立依附元件的有向圖，模擬應用程式中的資料流。

Angular 可以使用信號中的通知，瞭解需要進行變更偵測的元件，或執行您定義的效果函式。

將 `superSecretMessage` 轉換為 `signal()`

`superSecretMessage` 是 `MessageService` 中的值，定義玩家解碼的秘密訊息。目前這個值不會通知應用程式變更，因此「Customize」按鈕會失效。您可以使用信號解決這個問題。

將 `superSecretMessage` 設為信號，即可在訊息變更時通知應用程式中需要瞭解這項資訊的部分。在對話方塊中自訂訊息時，您會設定信號，以使用新訊息更新應用程式的其餘部分。

如要定義第一個信號，請在每個檔案的 `TODO(1): Define your first signal()` 註解下方執行下列步驟：

1. 在 `service.message.ts` 檔案中，使用 Signals 程式庫將 `superSecretMessage` 設為反應式：

`src/app/secret-message/service.message.ts`

```
superSecretMessage = signal(  
  'Angular Signals are in developer preview in v16 today!'  
);
```

系統會自動提示您從 `@angular/core` 匯入 `signal`。如果重新整理頁面，先前參照 `superSecretMessage` 的位置可能會發生錯誤。這是因為您已將 `superSecretMessage` 的類型從 `string` 變更為 `SettableSignal<string>`。如要修正這個問題，請將所有 `superSecretMessage` 參照變更為使用 Signals API。在讀取值的位置，呼叫 Signal getter `superSecretMessage()`。無論您在何處寫入值，都請在 `SettableSignal` 上使用 `.set` API，為訊息設定新值。

2. 在 `secret-message.ts` 和 `service.message.ts` 檔案中，將所有 `superSecretMessage` 參照更新為 `superSecretMessage()`：

`src/app/secret-message/secret-message.ts`

```
// Before  
this.messages.superSecretMessage  
this.messages.superSecretMessage = message;  
  
// After  
this.messages.superSecretMessage()  
this.messages.superSecretMessage.set(message);
```

`src/app/secret-message/service.message.ts`

```
// Before  
this.superSecretMessage  
  
// After  
this.superSecretMessage()
```


探索其他兩個信號

- 請注意，應用程式中還有另外兩個信號：

src/app/cipher/service.cipher.ts

```
cipher = signal(this.createNewCipherKey());
decodedCipher = signal<CipherKey[]>([]);
```

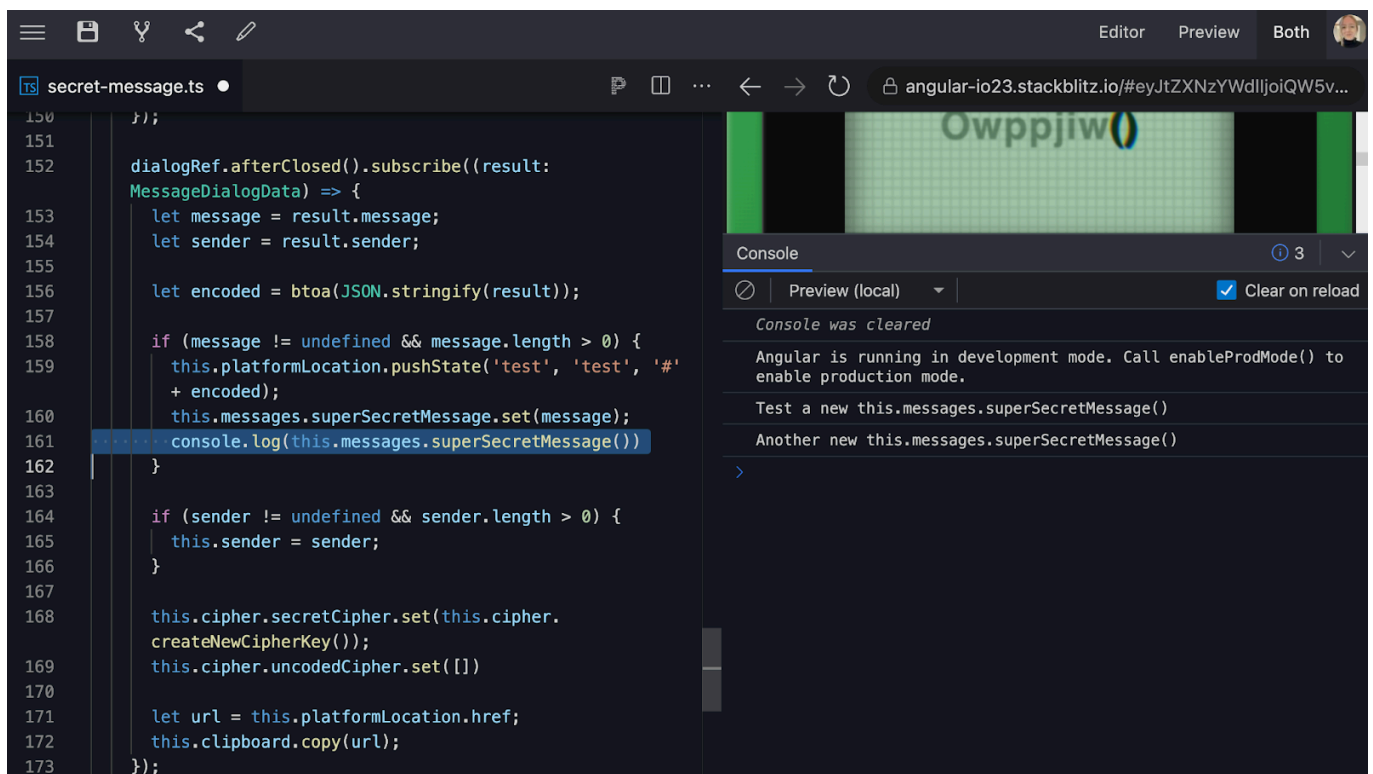
CipherService 會定義 cipher 信號，也就是隨機產生的對應項，將字母表中的一個字母對應到新的 cipher 字母。您可以使用這個函式將訊息打亂，並判斷玩家是否在鍵盤上找到相符的字元。

您也會收到成功解碼的鍵/值組合的 decodedCipher 信號，並在玩家解開密碼時新增至信號。

Angular 的 Signals 程式庫設計有一項獨特且強大的屬性，就是您可以在任何地方導入反應式。您在應用程式的服務中定義信號一次，即可在範本、元件、管道、其他服務或任何可編寫應用程式碼的地方使用信號。不會受限於元件範圍。

驗證變更

- 您還需要執行一個步驟，應用程式才能正常運作。目前請嘗試在應用程式的不同部分新增 console.log()，瞭解新的 superSecretMessage 設定方式。



步驟 5 · 約 5 分鐘

5. 定義第一個 computed()

在許多情況下，您可能會發現自己是從現有值衍生狀態。當相依值變更時，最好更新衍生狀態。

使用 `computed()`，您可以宣告式地表示信號，該信號的值是從其他信號衍生而來。

將 `solvedMessage` 轉換為 `computed()`

`solvedMessage` 會使用 `decodedCipher` 信號，將 `secretMessage` 值從編碼轉換為解碼。

這項功能非常實用，因為您可以根據另一個計算值衍生計算值，因此只要對應的反應式環境中任何信號發生變化，系統就會通知依附元件。

目前變更 `secretMessage`、`decodedCipher` 或 `superSecretMessage` 時，`solvedMessage` 不會更新。因此，當玩家解開密碼時，畫面不會更新。

建立 `solvedMessage` 計算值時，您會建立反應式環境，這樣一來，當您更新訊息或解開密碼時，就能從追蹤的依附元件衍生狀態更新。

如要將 `solvedMessage` 轉換為 `computed()`，請在每個檔案的 `TODO(2): Define your first computed()` 註解下方執行下列步驟：

1. 在 `service.message.ts` 檔案中，使用 Signals 程式庫將 `solvedMessage` 設為反應式：

`src/app/secret-message/service.message.ts`

```
solvedMessage = computed(() =>
  this.translateMessage(
    this.secretMessage(),
    this.cipher.decodedCipher()
  )
);
```

系統會自動提示您從 `@angular/core` 匯入 `computed`。如果重新整理頁面，先前參照 `solvedMessage` 的位置可能會發生錯誤。這是因為您已將 `superSecretMessage` 的類型從 `string` 變更為 `Signal<string>` (函式)。如要修正這個問題，請將 `solvedMessage` 的所有參照變更為 `solvedMessage()`。

2. 在 `secret-message.ts` 檔案中，將所有 `solvedMessage` 參照更新為 `solvedMessage()`：

`src/app/secret-message/secret-message.ts`

```
// Before
<span *ngFor="let char of this.messages.solvedMessage.split(''); index as i;"
[class.unsolved]="this.messages.solvedMessage[i] !== this.messages.superSecretMessage()[i]" >
{{ char }}</span>

// After
<span *ngFor="let char of this.messages.solvedMessage().split(''); index as i;"
[class.unsolved]="this.messages.solvedMessage()[i] !== this.messages.superSecretMessage()[i]"
>{{ char }}</span>
```

請注意，`solvedMessage` 與 `superSecretMessage` 不同，並非 `SettableSignal`，因此您無法直接變更其值。而是會在任一依附元件信號（`secretMessage` 和 `decodedCipher`）更新時，保持最新值。

探索其他兩個 `computed()` 函式

- 請注意，應用程式中還有兩個計算值：

`src/app/secret-message/service.message.ts`

```
secretMessage = computed(() =>
  this.translateMessage(
    this.superSecretMessage(),
    this.cipher.cipher()
  )
);
```

`src/app/cipher/service.cipher.ts`

```
unsolvedAlphabet = computed(() =>
  ALPHABET.filter(
    (letter) => !this.decodedCipher().find((guess) => guess.value === letter)
  )
);
```

`MessageService` 定義 `secretMessage` 計算結果，`superSecretMessage` 則是由 `cipher` 編碼，玩家必須解開這個問題。

`CipherService` 會定義 `unsolvedAlphabet` 計算結果，也就是玩家尚未解開的所有字母清單，這份清單衍生自 `decodedCipher` 中已解開的密碼金鑰清單。

驗證變更

現在 `superSecretMessage` 是信號，而 `solvedMessage` 是計算值，應用程式應該可以正常運作！測試遊戲功能：

1. 將 `LetterGuessComponent` 拖曳至 `CipherComponent` 中的 `LetterKeyComponent`，逐步解開密碼並解讀秘密訊息。 `LetterKeyComponent`
2. 解碼的密文越多，`SecretMessageComponent` 就會越完整。
3. 按一下「自訂」即可變更「寄件者」和「訊息」，然後按一下「建立並複製網址」，即可在畫面上查看值和網址變更。
4. 額外好康：將網址複製並貼到新分頁，或分享給朋友，看看網址中儲存了哪些寄件者和訊息資訊。

GOOGLE I/O 23

*Decipher a secret message from
the Angular Team:*

Angugaq
Dragnagd aqb rn
ibobgjybq
yqborbl rn o16
zjiam!

Customize



6. 新增第一個 effect()

有時您可能會希望在信號有新值時發生某件事。透過 `effect()`，您可以排定及執行處理常式函式，以回應信號變更。

在密碼解開時加入五彩碎紙

現在應用程式已可正常運作，您可以新增一些有趣的功能，例如在密碼破解且解碼秘密訊息時加入五彩碎紙。

如要新增五彩碎紙，請在 `TODO(3): Add your first effect()` 註解下方執行下列步驟：

1. 在 `cipher.ts` 檔案中，排定在解碼訊息時新增五彩碎紙的效果：

`src/app/cipher/cipher.ts`

```
import * as confetti from 'canvas-confetti';

ngOnInit(): void {
  ...

  effect(() => {
    if (this.messages.superSecretMessage() === this.messages.solvedMessage()) {
      var confettiCanvas = document.getElementById('confetti-canvas');
      confetti.create()(confettiCanvas, { particleCount: 100 });
    }
  });
}
```

請注意，這項效果取決於信號和計算值：`this.messages.superSecretMessage()` 和 `this.messages.solvedMessage()`。

Effect 可協助您在反應式環境中安排彩帶函式，以便追蹤並重新評估依附元件的更新時間。

驗證變更

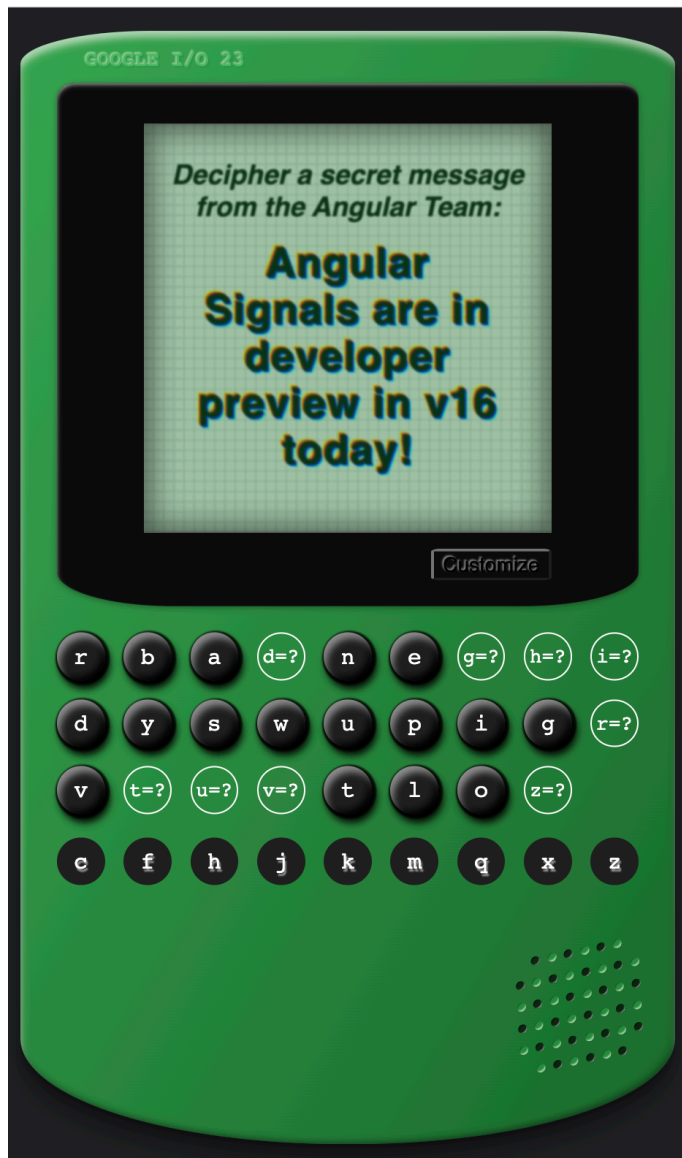
- 請嘗試解密 (提示：您可以將訊息改為較短的內容，加快測試速度！)。系統會顯示彩色碎紙，祝賀你獲得第一個 `effect()`！



步驟 7 · 約 1 分鐘

7. 恭喜！

現在可以開始解碼並分享秘密訊息了！想傳送訊息給 Angular 團隊嗎？在社群媒體上標記 @Angular，我們就會解碼！🎉



Angular 工具箱現在有三種新的反應式基本型別，可簡化開發作業，並預設建構速度更快的應用程式。

瞭解詳情

歡迎參考下列程式碼研究室：

- [開始使用獨立元件](#)
- [使用 Angular 和 Firebase 建構網頁應用程式](#)

請參閱下列資料：

- [Angular.io](#)
- [使用信號重新思考反應式設計](#) (2023 年 Google I/O 大會)
- [Angular 最新動態](#) (2023 年 Google I/O 大會)